

CITYRIDE 360

Complete Microsoft Fabric Data Engineering Capstone

A step-by-step project workbook for Exam DP-700

Build one solution that deliberately exercises the current exam blueprint: workspace management, lifecycle, security, orchestration, batch and streaming ingestion, SQL/PySpark/KQL transformations, monitoring, troubleshooting, and optimization.

Version: September 2026

How to use this workbook

CityRide is not an official Microsoft project. It is a custom capstone assembled around the official DP-700 skills measured and Microsoft Learning labs. Every stage tells you what to build, which official lab to consult, the code to write, what to verify, and the exam decision you should be able to explain afterwards.

Track	Recommended use	Time
Fast exam pass	Build stages 1-10, 13-15; read admin-only sections	10-14 hours
Full capstone	Complete every stage and every break/fix task	16-22 hours
Revision mode	Rebuild from the acceptance checks without reading the implementation steps	6-8 hours

Exam focus: The official blueprint says candidates should manipulate and transform data using SQL, PySpark, and KQL, and weights the three major domains at 30-35% each. This workbook intentionally gives all three languages meaningful work.

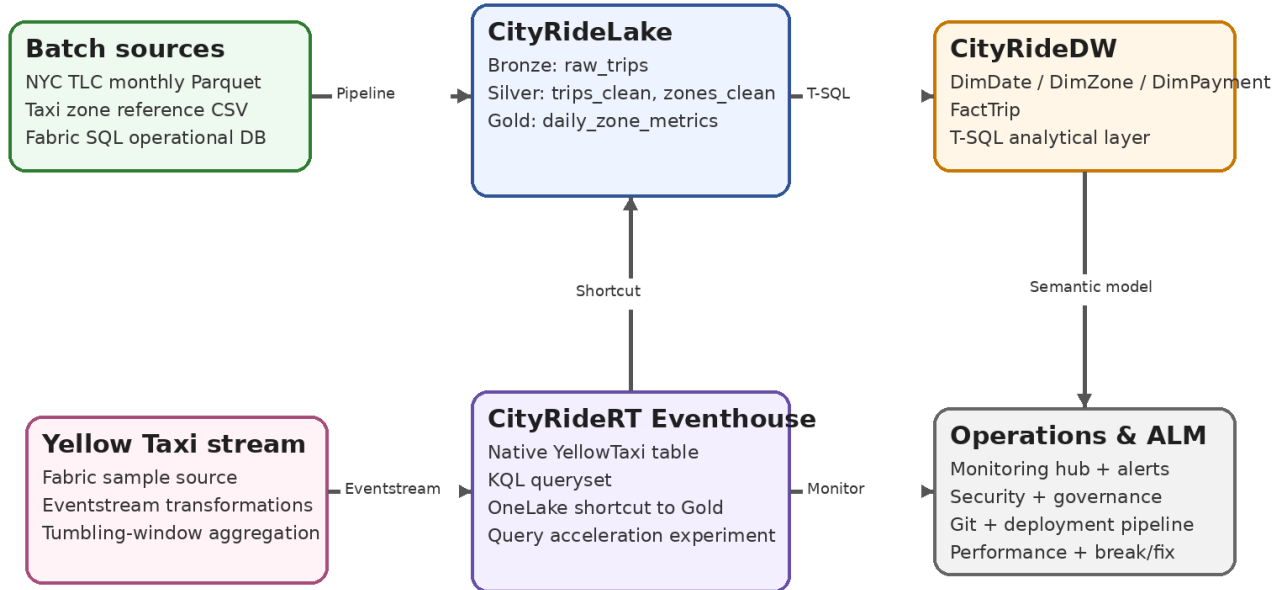
Prerequisites

- A Microsoft Fabric workspace assigned to a Fabric capacity or an active Fabric trial.
- Workspace Admin rights for most exercises. Some domain, audit, sensitivity-label, and tenant/delegated settings require additional organizational permissions; those steps are marked admin-dependent.
- A GitHub or Azure DevOps repository if you want to complete the lifecycle section.
- Optional: a second test identity or security group to validate row-level/column-level security from a restricted user perspective.
- Use the public NYC TLC Yellow Taxi 2024-01 and 2024-02 Parquet files for batch data. Fabric's built-in Yellow Taxi sample is used for the streaming path.

Project naming convention

Item type	Name
Workspace	DP700-CityRide-DEV
Lakehouse	CityRideLake
Warehouse	CityRideDW
Fabric SQL database	CityRideOps
Pipeline	pl_cityride_ingest
Dataflow Gen2	df_zones_clean
Notebook	nb_trip_transform
Streaming notebook	nb_trip_stream
Eventstream	es_cityride_live
Eventhouse	CityRideRT
KQL queryset	qs_cityride_live
Semantic model	CityRideModel
Deployment workspaces	DP700-CityRide-TEST / DP700-CityRide-PROD

Architecture and completion criteria



CityRide 360 - DP-700 capstone architecture

The project uses a lakehouse for open Delta data engineering, a warehouse for relational dimensional analytics, a Fabric SQL database for an operational/mirroring exercise, and an Eventhouse for real-time analytics. Orchestration, security, lifecycle, monitoring, and optimization are layered across the solution.

Definition of done

- Historical taxi data is ingested by a parameterized pipeline and processed through Bronze, Silver, and Gold Delta tables.
- Reference zone data is transformed with Dataflow Gen2 and joined in PySpark.
- A dimensional Warehouse is populated with T-SQL and supports analytical/window queries.
- An operational Fabric SQL database contains driver/vehicle data and its automatically mirrored OneLake copy is explored.
- A Yellow Taxi sample stream flows through Eventstream to Eventhouse; KQL performs filters, aggregations, and time windows.
- A Fabric notebook reads the Eventstream with Spark Structured Streaming and writes a streaming aggregate to Delta.
- An Eventhouse OneLake shortcut points to a Gold Delta table; you compare standard and accelerated shortcut querying.
- Security/governance, Git/deployment, monitoring, break/fix, and performance labs are completed and documented.

DP-700 blueprint coverage matrix

Current skill objective	CityRide coverage
-------------------------	-------------------

Configure Spark workspace settings	Stage 1, 14
Configure domain workspace settings	Stage 1
Configure OneLake workspace settings	Stage 1
Configure Apache Airflow workspace settings	Stage 1
Configure version control	Stage 12
Implement database projects	Stage 12
Create/configure deployment pipelines	Stage 12
Workspace/item access controls	Stage 11
Row/column/object/folder-file controls	Stage 11
Dynamic data masking	Stage 11
Sensitivity labels	Stage 11
Endorse items	Stage 11
Fabric audit logs	Stage 11
OneLake security	Stage 11
Choose Dataflow Gen2 vs pipeline vs notebook	Stages 3-5
Schedules and event-based triggers	Stage 3
Notebook/pipeline parameters and dynamic expressions	Stage 3
Full and incremental loads	Stage 3
Prepare dimensional data	Stages 5-6
Streaming load pattern	Stages 8-9
Choose an appropriate data store	Stages 2,6,8
Choose Dataflows/notebooks/KQL/T-SQL	Stages 4-8
OneLake shortcuts	Stage 10
Mirroring	Stage 7
Pipeline ingestion	Stage 3
PySpark/SQL/KQL transformations	Stages 5,6,8
Denormalize/group/aggregate	Stages 5-6
Duplicate/missing/late data	Stages 5,9
Choose streaming engine	Stages 8-9
Native RTI table vs OneLake shortcut	Stage 10
Query acceleration vs standard shortcut	Stage 10
Eventstreams	Stage 8
Spark Structured Streaming	Stage 9
KQL processing and windowing	Stage 8
Monitor ingestion/transformation/model refresh	Stage 13
Configure alerts	Stage 13
Resolve pipeline/Dataflow/notebook/Eventhouse/Eventstream/T-SQL/sh ortcut errors	Stage 13
Optimize Lakehouse/pipeline/warehouse/Eventstream/Eventhouse/Spark /query performance	Stage 14

Stage 1 - Workspace, settings, and engineering decisions

Exam focus: Configure Microsoft Fabric workspace settings; choose solution components; understand lifecycle/security boundaries.

Microsoft references to consult

[Official DP-700 interactive exercises](#) - Course filter -> DP-700 lists the 15 core labs used throughout this workbook.

[Create a Microsoft Fabric Lakehouse lab](#)

[Workspace administration settings \(Spark\)](#)

[Configure resource profiles in Fabric](#)

[Domains in Microsoft Fabric](#)

[Apache Airflow Job workspace settings](#)

Build steps

1. Create workspace DP700-CityRide-DEV on a Fabric-enabled capacity. Give it a description such as "DP-700 CityRide capstone".
2. Open Workspace settings > Data Engineering/Science > Spark settings. Record the current pool, autoscale, executor, and runtime-related defaults. Do not blindly increase compute; the goal is to know where these controls live and when a custom pool/resource profile is justified.
3. If available to your role, inspect Workspace settings > Delegated settings > OneLake settings. Record which OneLake controls are delegated to the workspace. If nothing is delegated, note that this is tenant/admin dependent rather than treating the absence as an error.
4. Assign the workspace to a Transportation domain if you have domain contributor/admin rights. Otherwise write down the navigation and required role.
5. Inspect Workspace settings > Data Factory > Apache Airflow Runtime Settings. Compare Starter Pool versus Custom Pool. For a study workspace, keep Starter Pool unless you explicitly want to practice custom Airflow compute.
6. Create CityRideLake (Lakehouse), CityRideDW (Warehouse), CityRideOps (SQL database), and CityRideRT (Eventhouse). Keep them in the same DEV workspace to make cross-item exercises simpler.

Write this architecture decision record

Need	Chosen Fabric item	Why
Raw/curated batch data	Lakehouse	Delta/OneLake, Spark-first engineering, open format
Dimensional relational serving	Warehouse	T-SQL, star schema, warehouse monitoring/security
Operational driver/vehicle records	Fabric SQL database	OLTP-style source plus automatic mirroring into OneLake
High-rate timestamped stream	Eventhouse	KQL-native real-time analytics
Batch orchestration	Pipeline	Copy + notebook + Dataflow activities, parameters/triggers
Low-code reference cleanup	Dataflow Gen2	Power Query transformations

Custom transformations	Notebook	PySpark and Structured Streaming
------------------------	----------	----------------------------------

Exam focus: Be ready for scenario questions: "Which item should I choose and why?" DP-700 is not only a button-location exam.

Stage 2 - Lakehouse and source data

Exam focus: Choose a data store; create/manage a Lakehouse; understand Files versus managed Delta tables and shortcuts.

Primary lab

[Create a Microsoft Fabric Lakehouse](#)

[NYC TLC Trip Record Data](#) - Official source for monthly Yellow Taxi trip records and the Taxi Zone Lookup table.

Download or pipeline-source these files

TEXT

```
https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet
https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-02.parquet
https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
```

Use January and February 2024 for a stable two-load exercise. One month is roughly millions of rows, enough to make Spark and optimization exercises realistic without requiring a huge dataset.

Create the lakehouse layout

TEXT

```
Files/
  landing/trips/
  landing/reference/
  quarantine/
Tables/
  bronze_trips
  silver_trips
  silver_zones
  gold_daily_zone_metrics
  gold_payment_metrics
  stream_trip_windows
```

7. In CityRideLake, create the landing/trips, landing/reference, and quarantine folders under Files.
8. Use Lakehouse explorer to review Files versus Tables. Remember: Files are arbitrary OneLake objects; Lakehouse Tables are Delta tables registered for SQL/Spark access.
9. Open the SQL analytics endpoint and confirm that no curated tables exist yet. You will return here after the Spark transformations.

Acceptance check

- CityRideLake exists.
- You can explain Files vs Tables vs a shortcut without using the UI.
- You have the two monthly trip URLs and zone lookup URL recorded.

Stage 3 - Parameterized batch ingestion: full and incremental

Exam focus: Pipelines, Copy Data, full/incremental loading patterns, schedules/event triggers, parameters, dynamic expressions, orchestration.

Primary lab

[Ingest data with a pipeline in Microsoft Fabric](#)

Create pipeline parameters

Parameter	Type	Example
p_year_month	String	2024-01
p_full_load	Boolean	true
p_target_folder	String	landing/trips
p_table_name	String	dbo.bronze_trips

Create a Copy Data activity using an anonymous HTTP connection to <https://d37ci6vzurychx.cloudfront.net> and a Lakehouse destination.

Dynamic expressions

FABRIC EXPRESSION

```
@concat('/trip-data/yellow_tripdata_', pipeline().parameters.p_year_month, '.parquet')
```

FABRIC EXPRESSION

```
@concat('yellow_tripdata_', pipeline().parameters.p_year_month, '.parquet')
```

FABRIC EXPRESSION

```
@concat(pipeline().parameters.p_target_folder, '/', pipeline().parameters.p_year_month)
```

Pipeline shape

TEXT

```
Copy monthly Parquet
|
v
Notebook: nb_trip_transform (base parameter p_year_month)
|
v
Optional Dataflow Gen2: df_zones_clean (first run / reference refresh)
|
v
Validation / success path
```

- First run: set p_year_month=2024-01 and p_full_load=true. Copy January to Files/landing/trips/2024-01/.
- Add a Notebook activity after Copy Data and pass p_year_month to a notebook parameter cell. The notebook is built in Stage 5.
- Second run: set p_year_month=2024-02 and p_full_load=false. This is the project's file-based incremental load: only the newly published month is copied and appended/merged into curated tables.
- Create a schedule trigger for a harmless practice cadence (for example daily while studying), inspect it, then disable it if you do not want recurring capacity consumption.

14. Inspect event-based trigger options and write down when you would prefer an event trigger: when a file arrival or external event should initiate work rather than a fixed clock schedule.
15. Add a deliberate failure by setting p_year_month=2099-99. Run, inspect the failed Copy activity output, then restore a valid month.

Optional control-table watermark pattern

For a row-level incremental pattern, create a tiny control table in CityRideDW or CityRideOps. This is useful exam practice even though the TLC source files themselves are monthly snapshots.

SQL

```
CREATE TABLE dbo.LoadWatermark (
    PipelineName varchar(100),
    LastSuccessfulWatermark datetime2
);

INSERT INTO dbo.LoadWatermark
VALUES ('pl_cityride_ingest', '2024-01-01T00:00:00');
```

Exam focus: Know the difference between a full load, incremental file load, incremental watermark load, and CDC/streaming. The optimal pattern depends on how the source exposes change.

Stage 4 - Dataflow Gen2 for reference data

Exam focus: Choose Dataflow Gen2 versus notebook/pipeline/T-SQL/KQL; perform low-code transformation; orchestrate a dataflow.

Primary lab

[Create and use Dataflows \(Gen2\) in Microsoft Fabric](#)

Build df_zones_clean

16. Create a Dataflow Gen2 named df_zones_clean.
17. Connect to taxi_zone_lookup.csv. If your tenant blocks a direct Web source, upload it to Files/landing/reference and use the Lakehouse source instead.
18. Keep LocationID, Borough, Zone, and service_zone.
19. Set LocationID to whole number; trim text; replace null/blank Borough or Zone values with Unknown; remove duplicate LocationID rows.
20. Rename service_zone to ServiceZone for consistent downstream naming.
21. Set destination to CityRideLake table dbo.silver_zones (replace mode for this small reference table).
22. Add the Dataflow activity to the batch pipeline and connect it before the Spark notebook when p_full_load=true, or run it independently as a reference-data refresh.

Why Dataflow Gen2 here?

Option	Would it work?	Why/why not for this task
Dataflow Gen2	Yes - chosen	Small tabular reference cleanup; visual Power Query is fast and maintainable
Notebook	Yes	Better when transformations are code-heavy, large-scale, reusable Spark logic
Pipeline only	Partially	Best for orchestration/movement; not the

		primary transformation engine for complex shaping
T-SQL	Yes after landing	Best when data already sits in a SQL-accessible relational layer
KQL	Possible but wrong fit	Optimized for event/time-series and Eventhouse workloads, not this small batch reference cleanse

Stage 5 - PySpark medallion transformations

Exam focus: PySpark transformations; medallion architecture; Delta; denormalization; grouping/aggregation; duplicates/missing data; parameters.

Primary labs

[Analyze data with Apache Spark](#)

[Use Delta tables in Apache Spark](#)

[Create a medallion architecture in a Microsoft Fabric lakehouse](#)

Notebook parameter cell

PYTHON

```
p_year_month = "2024-01"
p_full_load = True
```

Toggle this cell as a parameter cell so the pipeline can override the values.

Load the monthly Parquet and persist Bronze

PYTHON

```
from pyspark.sql import functions as F
from pyspark.sql import types as T

source_path = f"Files/landing/trips/{p_year_month}/*.parquet"
raw = spark.read.parquet(source_path)

bronze = (raw
    .withColumn("source_month", F.lit(p_year_month))
    .withColumn("ingested_at", F.current_timestamp()))

mode = "overwrite" if p_full_load else "append"
(bronze.write
    .format("delta")
    .mode(mode)
    .option("overwriteSchema", "true")
    .saveAsTable("dbo.bronze_trips"))
```

Clean Silver

PYTHON

```

from pyspark.sql import functions as F

b = spark.table("dbo.bronze_trips")

silver = (b
    .withColumn("pickup_ts", F.to_timestamp("tpep_pickup_datetime"))
    .withColumn("dropoff_ts", F.to_timestamp("tpep_dropoff_datetime"))
    .withColumn("pickup_date", F.to_date("tpep_pickup_datetime"))
    .withColumn("trip_duration_min",
        (F.unix_timestamp("tpep_dropoff_datetime") -
         F.unix_timestamp("tpep_pickup_datetime")) / 60.0)
    .withColumn("passenger_count", F.coalesce(F.col("passenger_count"), F.lit(1)))
    .filter(F.col("pickup_ts").isNotNull())
    .filter(F.col("dropoff_ts").isNotNull())
    .filter(F.col("trip_distance") >= 0)
    .filter(F.col("total_amount") >= 0)
    .filter(F.col("trip_duration_min").between(0, 24*60))
    .dropDuplicates([
        "VendorID", "tpep_pickup_datetime", "tpep_dropoff_datetime",
        "PULocationID", "DOLocationID", "total_amount"
    ]))

silver.write.format("delta").mode("overwrite").saveAsTable("dbo.silver_trips")

```

Quarantine invalid rows

PYTHON

```

invalid = (b
    .filter(
        F.col("tpep_pickup_datetime").isNull() |
        F.col("tpep_dropoff_datetime").isNull() |
        (F.col("trip_distance") < 0) |
        (F.col("total_amount") < 0)
    ))

invalid.write.mode("overwrite").parquet("Files/quarantine/trips")

```

Join zones and build Gold

PYTHON

```
trips = spark.table("dbo.silver_trips")
zones = spark.table("dbo.silver_zones")

pu = zones.select(
    F.col("LocationID").alias("PULocationID"),
    F.col("Borough").alias("PickupBorough"),
    F.col("Zone").alias("PickupZone"))

do = zones.select(
    F.col("LocationID").alias("DOLocationID"),
    F.col("Borough").alias("DropoffBorough"),
    F.col("Zone").alias("DropoffZone"))

enriched = (trips
    .join(F.broadcast(pu), "PULocationID", "left")
    .join(F.broadcast(do), "DOLocationID", "left")
    .fillna({"PickupBorough": "Unknown", "PickupZone": "Unknown",
            "DropoffBorough": "Unknown", "DropoffZone": "Unknown"}))

gold = (enriched
    .groupBy("pickup_date", "PULocationID", "PickupBorough", "PickupZone")
    .agg(
        F.count("*").alias("TripCount"),
        F.round(F.sum("total_amount"), 2).alias("Revenue"),
        F.round(F.avg("fare_amount"), 2).alias("AvgFare"),
        F.round(F.avg("trip_distance"), 2).alias("AvgDistance"),
        F.round(F.avg("trip_duration_min"), 2).alias("AvgDurationMin"),
        F.sum("passenger_count").alias("Passengers")))

gold.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_daily_zone_metrics")
```

Quality checks you should run

PYTHON

```
# Duplicate rate before/after
print("bronze", spark.table("dbo.bronze_trips").count())
print("silver", spark.table("dbo.silver_trips").count())

# Missing-zone checks
spark.table("dbo.gold_daily_zone_metrics").filter("PickupZone = 'Unknown'")
    .groupBy("PULocationID").count().orderBy(F.desc("count")).show(20)

# Negative amount should be zero in Silver
assert spark.table("dbo.silver_trips").filter("total_amount < 0").count() == 0
```

Exam focus: This stage deliberately performs both normalization (clean Silver) and denormalization (zone attributes embedded into an enriched analytical dataset). Be able to explain the trade-off.

Stage 6 - Dimensional Warehouse and T-SQL

Exam focus: Prepare/load a dimensional model; T-SQL transformations; joins, aggregations, window functions; anomaly checks; warehouse query patterns.

Primary labs

[Analyze data in a data warehouse](#)

[Load data into a warehouse using T-SQL](#)

[Query a data warehouse in Microsoft Fabric](#)

[Design and implement a dimensional model \(supplement\)](#)

Create the star schema

SQL

```

CREATE TABLE dbo.DimDate (
    DateKey int NOT NULL,
    FullDate date NOT NULL,
    CalendarYear smallint NOT NULL,
    CalendarMonth tinyint NOT NULL,
    MonthName varchar(12) NOT NULL,
    DayOfWeek tinyint NOT NULL,
    DayName varchar(12) NOT NULL
);

CREATE TABLE dbo.DimZone (
    ZoneKey int NOT NULL,
    Borough varchar(100),
    ZoneName varchar(150),
    ServiceZone varchar(100)
);

CREATE TABLE dbo.DimPayment (
    PaymentKey int NOT NULL,
    PaymentName varchar(50) NOT NULL
);

CREATE TABLE dbo.FactTrip (
    PickupDateKey int NOT NULL,
    PickupZoneKey int,
    DropoffZoneKey int,
    PaymentKey int,
    PassengerCount int,
    TripDistance decimal(12,2),
    TripDurationMin decimal(12,2),
    FareAmount decimal(12,2),
    TipAmount decimal(12,2),
    TotalAmount decimal(12,2)
);

```

Load dimensions

SQL

```

INSERT INTO dbo.DimPayment VALUES
(0, 'Unknown'), (1, 'Credit card'), (2, 'Cash'), (3, 'No charge'),
(4, 'Dispute'), (5, 'Unknown/other'), (6, 'Voided trip');

INSERT INTO dbo.DimZone (ZoneKey, Borough, ZoneName, ServiceZone)
SELECT DISTINCT LocationID, Borough, Zone, ServiceZone
FROM CityRideLake.dbo.silver_zones;

```

SQL

```

INSERT INTO dbo.DimDate
SELECT DISTINCT
    YEAR(pickup_date) * 10000 + MONTH(pickup_date) * 100 + DAY(pickup_date) AS DateKey,
    pickup_date,
    YEAR(pickup_date), MONTH(pickup_date), DATENAME(month, pickup_date),
    DATEPART(weekday, pickup_date), DATENAME(weekday, pickup_date)
FROM CityRideLake.dbo.silver_trips s
WHERE NOT EXISTS (
    SELECT 1 FROM dbo.DimDate d WHERE d.FullDate = s.pickup_date
);

```

Load the fact table from the Lakehouse SQL endpoint

SQL

```
TRUNCATE TABLE dbo.FactTrip;

INSERT INTO dbo.FactTrip
SELECT
    YEAR(s.pickup_date) * 10000 + MONTH(s.pickup_date) * 100 + DAY(s.pickup_date),
    s.PULocationID,
    s.DOLocationID,
    COALESCE(s.payment_type, 0),
    CAST(s.passenger_count AS int),
    CAST(s.trip_distance AS decimal(12,2)),
    CAST(s.trip_duration_min AS decimal(12,2)),
    CAST(s.fare_amount AS decimal(12,2)),
    CAST(s.tip_amount AS decimal(12,2)),
    CAST(s.total_amount AS decimal(12,2))
FROM CityRideLake.dbo.silver_trips AS s;
```

Analytical query + SQL window function

SQL

```
WITH DailyRevenue AS (
    SELECT
        PickupDateKey,
        PickupZoneKey,
        COUNT(*) AS TripCount,
        SUM(TotalAmount) AS Revenue
    FROM dbo.FactTrip
    GROUP BY PickupDateKey, PickupZoneKey
)
SELECT
    PickupDateKey,
    PickupZoneKey,
    TripCount,
    Revenue,
    SUM(Revenue) OVER (
        PARTITION BY PickupZoneKey
        ORDER BY PickupDateKey
        ROWS UNBOUNDED PRECEDING
    ) AS RunningRevenue
FROM DailyRevenue
ORDER BY PickupZoneKey, PickupDateKey;
```

Consistency checks

SQL

```
SELECT COUNT(*) AS NegativeAmounts
FROM dbo.FactTrip
WHERE TotalAmount < 0 OR FareAmount < 0;

SELECT COUNT(*) AS MissingZoneKeys
FROM dbo.FactTrip f
LEFT JOIN dbo.DimZone z ON f.PickupZoneKey = z.ZoneKey
WHERE z.ZoneKey IS NULL;
```

Exam focus: Know why a star schema can improve usability and analytical query patterns even when the Lakehouse already contains curated Delta tables.

Stage 7 - Fabric SQL database and mirroring

Exam focus: Implement mirroring; distinguish operational SQL storage from analytical stores; query mirrored data through OneLake/SQL analytics.

References

[Work with SQL Database in Microsoft Fabric lab](#)

[Mirroring Fabric SQL database in Microsoft Fabric](#)

[Get started with mirroring in Fabric](#)

Create operational tables in CityRideOps

SQL

```
CREATE TABLE dbo.Driver (
    DriverId int PRIMARY KEY,
    FullName varchar(100) NOT NULL,
    Email varchar(200),
    LicenseNumber varchar(50),
    Active bit NOT NULL,
    ModifiedDate datetime2 NOT NULL DEFAULT sysdatetime()
);

CREATE TABLE dbo.Vehicle (
    VehicleId int PRIMARY KEY,
    RegistrationNumber varchar(30) NOT NULL,
    VehicleType varchar(30),
    ModifiedDate datetime2 NOT NULL DEFAULT sysdatetime()
);

INSERT dbo.Driver (DriverId, FullName, Email, LicenseNumber, Active)
VALUES
(1, 'Amina Diallo', 'amina@example.test', 'LIC-1001', 1),
(2, 'Marco Silva', 'marco@example.test', 'LIC-1002', 1),
(3, 'Lena Martin', 'lena@example.test', 'LIC-1003', 0);

INSERT dbo.Vehicle (VehicleId, RegistrationNumber, VehicleType)
VALUES (101, 'CITY-101', 'Sedan'), (102, 'CITY-102', 'EV'), (103, 'CITY-103', 'Van');
```

23. Open the SQL database SQL analytics endpoint and find the mirrored analytical representation.
24. Verify that the Fabric SQL database mirrors supported tables automatically into OneLake; no separate ETL pipeline is required for this built-in mirroring path.
25. Update one driver record in CityRideOps, then observe the analytical/mirrored side after replication catches up.
26. Write a note comparing mirroring with Copy Data: mirroring continuously replicates supported changes for analytics; Copy Data is an explicit orchestration/data movement activity.

Stage 8 - Eventstream, Eventhouse, and KQL

Exam focus: Choose a streaming engine; Eventstreams; KQL; native Eventhouse table; aggregations and time-window functions; real-time ingestion monitoring.

Primary labs and reference

[Get started with Real-Time Intelligence in Microsoft Fabric](#)

[Ingest real-time data with Eventstream in Microsoft Fabric](#)

[Add a sample data source to an Eventstream](#) - Confirms Yellow Taxi is an available built-in sample source.

Build the real-time path

27. Create/open CityRideRT Eventhouse and its KQL database.
28. Create es_cityride_live Eventstream.
29. Add Source > Sample data > Yellow Taxi. Preview the schema; you should see familiar taxi fields such as tpep_pickup_datetime, tpep_dropoff_datetime, trip_distance, and total_amount.
30. Add a Filter transformation for total_amount > 0.
31. Add a Group by transformation to calculate trip count or total revenue in a tumbling window (for example 1 minute). This is your visual Eventstream windowing exercise.
32. Add an Eventhouse destination. Create a native table called YellowTaxiNative. Publish the eventstream and wait for data to arrive.
33. Open a KQL queryset named qs_cityride_live.

KQL: health check and filtering

KQL

```
YellowTaxiNative
| where ingestion_time() > ago(10m)
| project tpep_pickup_datetime, tpep_dropoff_datetime,
    passenger_count, trip_distance, fare_amount, tip_amount, total_amount
| take 100
```

KQL: one-minute time buckets

KQL

```
YellowTaxiNative
| extend pickup_ts = todatetime(tpep_pickup_datetime)
| summarize
    Trips = count(),
    Revenue = sum(todouble(total_amount)),
    AvgFare = avg(todouble(fare_amount))
    by bin(pickup_ts, 1m)
| order by pickup_ts desc
```

KQL: useful anomaly query

KQL

```
YellowTaxiNative
| extend pickup_ts = todatetime(tpep_pickup_datetime),
    dropoff_ts = todatetime(tpep_dropoff_datetime)
| extend duration_min = datetime_diff('minute', dropoff_ts, pickup_ts)
| where duration_min < 0 or todouble(total_amount) < 0
| project pickup_ts, dropoff_ts, duration_min, total_amount
| take 50
```

Exam focus: KQL is optimized for high-volume event/time-series exploration. T-SQL can also query Eventhouse in some scenarios, but DP-700 expects you to be comfortable choosing KQL for real-time analytical work.

Stage 9 - Spark Structured Streaming

Exam focus: Process streaming data with Spark Structured Streaming; event-time windows; watermarks/late data; choose Eventstream/KQL/Spark roles.

Reference

[Query Eventstream from a notebook with Spark Structured Streaming](#)

[Explore sample data using Eventstream and Spark Structured Streaming](#)

Connect the Eventstream to nb_trip_stream

34. Create notebook nb_trip_stream and attach CityRideLake.
35. In the notebook Explorer choose Add data items > From Real-Time hub and add es_cityride_live.
36. Right-click the default/derived taxi stream and choose Read with Spark. Fabric generates the connection identifiers and a readStream scaffold.
37. Keep the generated Eventstream connection code. Replace only the downstream parsing/aggregation with logic matching the actual payload schema shown in your notebook preview.

The following pattern assumes the generated stream has been parsed to a DataFrame named events with timestamp and numeric taxi columns. If Fabric gives you raw Kafka value bytes, first parse the JSON using from_json and the schema you observe in Preview.

PYTHON

```
from pyspark.sql import functions as F

# Example once columns are parsed from the event payload
stream_clean = (events
    .withColumn("pickup_ts", F.to_timestamp("tpep_pickup_datetime"))
    .withColumn("total_amount_d", F.col("total_amount").cast("double"))
    .filter(F.col("pickup_ts").isNotNull())
    .filter(F.col("total_amount_d") >= 0))

windowed = (stream_clean
    .withWatermark("pickup_ts", "10 minutes")
    .groupBy(F.window("pickup_ts", "5 minutes"), F.col("PULocationID"))
    .agg(
        F.count("*").alias("TripCount"),
        F.sum("total_amount_d").alias("Revenue"),
        F.avg("trip_distance").alias("AvgDistance")))

query = (windowed.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "Files/checkpoints/cityride_trip_windows")
    .toTable("dbo.stream_trip_windows"))
```

Late-arriving-data thought exercise

- Event time is when the taxi trip occurred; ingestion time is when Fabric received the event.

- A watermark tells Spark how long to keep state for potentially late events. Events later than the watermark horizon can be dropped from stateful window results.
- Choose the watermark based on business tolerance and observed lateness, not an arbitrary number. Longer watermarks retain more state and can increase resource use.

Stage 10 - OneLake shortcuts and query acceleration in Real-Time Intelligence

Exam focus: Create/manage OneLake shortcuts; native Eventhouse table vs shortcut; standard vs query-accelerated shortcut.

References

[Discover and connect to data in OneLake lab](#)

[Create OneLake shortcuts in a KQL database](#)

[Query acceleration for OneLake shortcuts - overview](#)

[Enable query acceleration over OneLake shortcuts](#)

Experiment

38. In the CityRideRT KQL database choose New > OneLake shortcut.
39. Choose Microsoft OneLake and point the shortcut to CityRideLake > dbo.gold_daily_zone_metrics.
40. Create it first with acceleration OFF. Query it with `external_table()`. Record duration and scanned/returned data indicators available in the UI.
41. Enable query acceleration for the shortcut from Manage > Data policies. Set a caching period that covers the test data.
42. Run the same query several times and record the behavior. Do not claim a specific speed-up from a tiny dataset; the important part is the architectural trade-off.
43. Compare against YellowTaxiNative, which is physically ingested into Eventhouse and optimized for native KQL access.

KQL

```
external_table('gold_daily_zone_metrics')
| summarize Revenue = sum(todouble(Revenue)), Trips = sum(tolong(TripCount))
  by PickupBorough
| order by Revenue desc
```

Choice	Use when	Trade-off
Native Eventhouse table	High-performance real-time/event queries are primary	Ingestion/copy into Eventhouse must be maintained
Standard OneLake shortcut	Data should remain in OneLake and query latency is acceptable	No duplicate copy, but external queries can be slower
Accelerated shortcut	Keep one logical copy in OneLake but require near-native KQL performance	Caching/acceleration policy and capacity cost; supported scenarios/limits apply

Stage 11 - Security and governance

Exam focus: Workspace/item security; RLS/CLS/object/folder-file controls; masking; sensitivity labels; endorsement; audit; OneLake security.

Primary labs

[Secure data access in Microsoft Fabric](#)

[Secure data in a data warehouse](#)

[Govern analytics data in Microsoft Fabric \(supplement\)](#)

Security personas

Persona	Expected access
Fabric admin / workspace admin	All DEV items, configuration
Data Engineer	Contributor-style access to engineering items
Operations Analyst	Read analytical Gold/Warehouse data, not raw PII
External/Restricted Analyst	Limited rows/columns; no raw operational identifiers

Warehouse dynamic data masking

SQL

```
CREATE TABLE dbo.DriverSecure (
    DriverId int,
    FullName varchar(100),
    Email varchar(200) MASKED WITH (FUNCTION = 'email()'),
    LicenseNumber varchar(50) MASKED WITH (FUNCTION = 'partial(0,"XXXX",4)')
);
```

Warehouse row-level security pattern

SQL

```
CREATE SCHEMA rls;
GO

CREATE FUNCTION rls.fn_zone_access(@AllowedUser varchar(256))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS access_result
WHERE @AllowedUser = USER_NAME();
GO

-- For hands-on validation, use a tiny exercise table that stores AllowedUser per row.
-- Then apply a FILTER PREDICATE to that column, following the official secure-warehouse lab.
```

Hands-on security checklist

44. Workspace: assign/inspect Admin, Member, Contributor, Viewer roles. Explain why workspace Viewer does not automatically mean unrestricted SQL permissions in every scenario.
45. Item: grant a test identity access to CityRideDW without giving broad edit rights to the whole workspace when possible.

46. Warehouse: practice dynamic data masking, RLS, column-level GRANT/DENY, and object-level permissions using the official secure-warehouse lab pattern.
47. Lakehouse: open Manage OneLake data access and create a OneLake security role that can read Gold tables while excluding Files/landing or Files/quarantine. Test with a second identity if available.
48. Governance: apply a sensitivity label to an item if your tenant has labels configured. Endorse/promote the Gold analytical item and inspect lineage.
49. Audit: if you have the required admin/compliance permissions, locate Fabric/Power BI activities in Microsoft Purview Audit. If you do not, document the prerequisite and know that audit is an organizational control rather than a normal workspace-member feature.

Exam focus: Do not conflate workspace roles, SQL permissions, OneLake security, and semantic model RLS. They protect different surfaces and can interact.

Stage 12 - Git, database projects, and deployment pipelines

Exam focus: Lifecycle management: version control, database projects, deployment pipelines, DEV/TEST/PROD promotion.

Primary lab and references

[Implement deployment pipelines in Microsoft Fabric](#)

[Get started with Git integration](#)

[Git integration for Fabric Warehouse development](#)

Git exercise

50. Create a repository such as dp700-cityride and connect DP700-CityRide-DEV through Workspace settings > Git integration.
51. Commit supported Fabric item definitions. Review which items are represented in source control and how the warehouse schema is stored.
52. Create a feature branch feature/add-tip-metric.
53. Add a Warehouse view or metric that calculates average tip percentage. Commit from Fabric or your supported development workflow.
54. Merge the feature branch into main, then update/sync the Fabric workspace from Git. Resolve any conflict only after comparing workspace versus branch changes.

SQL

```
CREATE OR ALTER VIEW dbo.vw_ZoneTipMetrics AS
SELECT
    PickupZoneKey,
    COUNT(*) AS TripCount,
    AVG(CASE WHEN FareAmount > 0 THEN TipAmount / FareAmount END) AS AvgTipRate
FROM dbo.FactTrip
GROUP BY PickupZoneKey;
```

Database project exercise

Inspect the Warehouse representation in Git/database-project form. If you have VS Code/SQL Database Projects tooling available, make one small schema change locally, build/validate it, commit it, and synchronize it back to the Fabric workspace. The goal is to understand schema-as-code and controlled deployment rather than treating a live Warehouse as the only source of truth.

Deployment pipeline exercise

55. Create DP700-CityRide-TEST and DP700-CityRide-PROD workspaces on Fabric capacity.
56. Create a deployment pipeline with DEV -> TEST -> PROD stages and assign the workspaces.
57. Deploy supported CityRide items to TEST. Inspect differences and deployment rules/parameters available for environment-specific values.
58. Make a small DEV change, redeploy to TEST, validate, then deploy to PROD.
59. Record which settings/secrets/connections should not be hard-coded in item logic and how environment-specific configuration should be handled.

Stage 13 - Monitoring, alerts, and systematic break/fix

Exam focus: Monitor ingestion/transformation/model refresh; configure alerts; identify/resolve pipeline, Dataflow, notebook, Eventhouse, Eventstream, T-SQL, and shortcut errors.

Primary labs

[Monitor Fabric activity in the monitoring hub](#)

[Monitor a data warehouse in Microsoft Fabric](#)

Monitoring hub exercise

60. Open Monitoring hub and filter by workspace/item type. Find successful and failed pipeline runs, Dataflow refreshes, and notebook/Spark jobs from this project.
61. Open a pipeline run and drill into activity input/output, duration, rows/files moved, and error details.
62. Open a notebook run and inspect Spark application/job/stage information and logs.
63. Create a minimal semantic model from CityRideDW or CityRideLake Gold data (for example a model with DimDate, DimZone, and FactTrip/gold metrics). Trigger a refresh where applicable and inspect refresh history/Monitoring hub so you have explicitly practiced the blueprint's semantic-model-refresh monitoring objective.
64. In Real-Time Intelligence, inspect Eventstream health/data insights and Eventhouse ingestion/query diagnostics.

Warehouse DMV/query-insights practice

SQL

```
SELECT * FROM sys.dm_exec_connections;
SELECT * FROM sys.dm_exec_sessions;
SELECT * FROM sys.dm_exec_requests;

SELECT TOP 50 *
FROM queryinsights.exec_requests_history
ORDER BY start_time DESC;
```

Create at least one real-time alert

Use Activator/alerting from a Real-Time query or event stream. Example business condition: alert when one-minute average fare exceeds a threshold you choose, or when TripCount falls below an expected threshold during a demo window. The value itself is not exam knowledge; knowing how to turn monitored data into an action is.

Break/fix matrix - do every row

Component	Introduce failure	Where to diagnose	Fix
Pipeline	Set p_year_month to a non-existent month	Copy activity error details, source URL, connection/auth, dynamic expression	Restore valid parameter; rerun
Dataflow Gen2	Change LocationID conversion so text errors	Refresh history / query step error	Fix type handling; refresh
Notebook	Reference a non-existent column	Notebook output + Spark logs	Correct schema/column; rerun
Eventstream	Point destination to wrong/nonexistent mapping or disconnect node	Eventstream Live view / destination status / data insights	Repair mapping/connection; republish
Eventhouse	Query/cast a field with wrong type or inspect ingestion failure	KQL error details / ingestion monitoring	Fix cast, mapping, or table schema
T-SQL	Misspell a column/table; run an invalid conversion	SQL error plus query insights/history	Fix query/schema
OneLake shortcut	Remove/alter access or point to wrong table (then restore)	Shortcut state and KQL external_table error	Restore security/source or recreate shortcut

Use this incident note template

TEXT

```
Symptom:
Affected item/run:
First diagnostic surface checked:
Error message / evidence:
Root cause:
Fix:
How I would prevent recurrence:
DP-700 concept tested:
```

Stage 14 - Performance optimization lab

Exam focus: Optimize Lakehouse tables, Spark, pipelines, Warehouse, Eventstream/Eventhouse, and query performance. Measure before/after; do not optimize by folklore.

References

[Optimize Delta Lake tables with V-Order](#)

[Performance guidelines in Fabric Data Warehouse](#)[Monitor a data warehouse lab](#)

Lakehouse optimization

First create/observe a table with multiple files. Record file count and a representative query duration. Then compact and optionally organize data. Exact gains depend on data volume and workload.

SPARK SQL

```
OPTIMIZE dbo.silver_trips;

-- When a frequent filter column is suitable for clustering in your test:
OPTIMIZE dbo.silver_trips ZORDER BY (pickup_date) VORDER;
```

- Explain the small-files problem.
- Explain that V-Order is a Parquet write/read optimization and Z-Order organizes data for skipping on selected columns.
- Do not run OPTIMIZE after every tiny write; maintenance frequency should match workload and file growth.

Spark optimization experiment

PYTHON

```
# Inspect partitioning
print(spark.table("dbo.silver_trips").rdd.getNumPartitions())

# Small dimension: compare regular join vs broadcast join
from pyspark.sql.functions import broadcast
fast_join = spark.table("dbo.silver_trips").join(
    broadcast(spark.table("dbo.silver_zones")),
    spark.table("dbo.silver_trips").PULocationID == spark.table("dbo.silver_zones").LocationID,
    "left")

fast_join.explain("formatted")
```

- Look for unnecessary shuffles.
- Use broadcast only when the dimension is actually small enough.
- Cache only reused intermediate data that benefits from caching; caching everything consumes memory and can hurt performance.
- Revisit Workspace Spark settings/resource profile only after identifying a compute bottleneck.

Warehouse optimization experiment

SQL

```
-- Baseline query
SELECT PickupZoneKey, COUNT(*) AS Trips, SUM(TotalAmount) AS Revenue
FROM dbo.FactTrip
WHERE PickupDateKey BETWEEN 20240101 AND 20240229
GROUP BY PickupZoneKey;

-- Inspect query history / long-running query views.
SELECT * FROM queryinsights.long_running_queries;
```

- Use appropriate data types and matching join-column types to avoid conversions.
- Let automatic statistics work, but know CREATE/UPDATE STATISTICS can be useful around large transformations/maintenance windows.

- Filter early and select only required columns.
- Use Query Insights and DMVs to identify actual bottlenecks rather than assuming the slowest-looking SQL is the cause.

Pipeline optimization

- Remove redundant copies/deletes.
- Parallelize independent activities when dependencies permit.
- Pass parameters rather than duplicate nearly identical pipelines.
- Avoid launching Spark for transformations that a simple low-code/SQL step can do more efficiently, and avoid forcing everything into Dataflow when code-scale Spark is more appropriate.

Eventhouse/Eventstream optimization

- Compare native Eventhouse ingestion to standard and accelerated OneLake shortcuts for the same type of query.
- Push simple filters/aggregations into Eventstream when reducing downstream event volume is useful.
- Choose sensible KQL time filters and project only needed columns.
- Use bin()/time windows appropriate to the decision latency; extremely fine windows increase result/state volume.

Stage 15 - Final exam simulation and project sign-off

***Exam focus:** Turn implementation experience into scenario reasoning. You should be able to rebuild or troubleshoot the solution without a recipe.*

30-minute rebuild challenge

65. From a blank workspace, create a Lakehouse, a parameterized HTTP-to-Lakehouse Copy pipeline, and a notebook parameter cell.
66. Read one taxi Parquet file with PySpark, clean negative/null rows, group by pickup zone/date, and write a Delta Gold table.
67. Create a Warehouse table and load/query curated data with T-SQL.
68. Create an Eventstream using Yellow Taxi sample data, route it to Eventhouse, and write a KQL one-minute aggregation.
69. Open Monitoring hub and locate the runs you just created.

Scenario questions you must answer aloud

- **Why use a Lakehouse rather than a Warehouse for the Bronze/Silver Spark path?**
- **When would you use a Warehouse instead of Lakehouse SQL analytics endpoint?**
- **Why use Dataflow Gen2 for zones but PySpark for trips?**
- **When is a pipeline incremental load better than mirroring?**
- **What is the difference between mirroring and a OneLake shortcut?**
- **Why ingest into Eventhouse natively instead of querying a OneLake shortcut?**
- **When would accelerated shortcuts be preferable to native ingestion?**
- **What problem does a Spark watermark solve, and what happens to very-late events?**

- **Where do you diagnose a Copy failure versus a Spark failure versus a KQL ingestion failure?**
- **How do workspace roles, OneLake security, SQL RLS/CLS, and semantic-model RLS differ?**
- **What evidence would you collect before changing Spark compute or running table optimization?**
- **Why is Git integration not the same thing as a deployment pipeline?**

Final sign-off checklist

Deliverable	Done
Workspace/Spark/domain/OneLake/Airflow settings inspected	☐
Git + database project + deployment pipeline practiced	☐
Workspace/item/SQL/OneLake security practiced	☐
Masking, sensitivity/endorsement/audit concepts covered	☐
Pipeline parameters, expressions, schedule/event triggers practiced	☐
Full + incremental batch load completed	☐
Dataflow Gen2 reference transform completed	☐
PySpark Bronze/Silver/Gold completed	☐
Duplicate/missing/invalid data handling completed	☐
Warehouse star schema + T-SQL load completed	☐
SQL window/aggregation queries completed	☐
Fabric SQL database mirroring verified	☐
Eventstream Yellow Taxi -> native Eventhouse completed	☐
KQL filtering/aggregation/windowing completed	☐
Spark Structured Streaming + watermark/window completed	☐
OneLake Eventhouse shortcut completed	☐
Standard vs accelerated shortcut compared	☐
Monitoring hub + Warehouse monitoring completed	☐
Semantic model refresh monitoring practiced	☐
Alert configured	☐
All break/fix scenarios completed	☐
Lakehouse/Spark/Warehouse/pipeline/RTI optimization experiments completed	☐

Suggested 3-day execution plan

Day	Build	Official labs to use
Day 1 - Batch foundation	Stages 1-6: workspace, Lakehouse, pipeline, Dataflow, PySpark, Warehouse	01, 02, 03, 03b, 04, 05, 06, 06a, 06b
Day 2 - Real time + security + ALM	Stages 7-12: SQL DB/mirroring, Eventstream/Eventhouse, Structured Streaming, shortcuts, security, Git/deployment	20, 07, 09, 25, 19, 06d, 21 + linked Microsoft Learn docs
Day 3 - Operations + exam rehearsal	Stages 13-15: monitoring, break/fix, optimization, rebuild challenge	18, 06c + performance docs + official DP-700 study guide

Official references and exact labs

The project is custom; the references below are the authoritative Microsoft materials used to map and validate each project stage. URLs were verified in September 2026. Microsoft can rename UI labels or update labs, so prefer the current page if a screenshot differs.

[DP-700 study guide - skills measured July 21, 2026](#)

[Microsoft Fabric interactive exercises - DP-700 course filter](#)[01 - Create a Microsoft Fabric Lakehouse](#)[02 - Analyze data with Apache Spark](#)[03 - Use Delta tables in Apache Spark](#)[03b - Create a medallion architecture](#)[04 - Ingest data with a pipeline](#)[05 - Create and use Dataflows Gen2](#)[06 - Analyze data in a data warehouse](#)[06a - Load data into a warehouse using T-SQL](#)[06b - Query a data warehouse](#)[06c - Monitor a data warehouse](#)[06d - Secure data in a data warehouse](#)[07 - Get started with Real-Time Intelligence](#)[09 - Ingest real-time data with Eventstream](#)[18 - Monitor Fabric activity in monitoring hub](#)[19 - Secure data access in Microsoft Fabric](#)[19b - Govern analytics data](#)[20 - Work with SQL Database in Microsoft Fabric](#)[21 - Implement deployment pipelines](#)[25 - Discover and connect to data in OneLake](#)[26 - Design and implement a dimensional model](#)[End-to-end Fabric tutorials](#)[Lakehouse end-to-end scenario](#)[Yellow Taxi sample source for Eventstream](#)[Spark Structured Streaming with Eventstream](#)[OneLake shortcuts in KQL database](#)[Query acceleration for OneLake shortcuts](#)

[Fabric SQL database mirroring](#)[Fabric Git integration](#)[Warehouse Git/database-project development](#)[Delta optimization and V-Order](#)[Warehouse performance guidelines](#)[NYC TLC Trip Record Data](#)

Appendix A - Command quick reference

PySpark transformations to remember

PYTHON

```

df.select(...)
df.filter(...) / df.where(...)
df.withColumn(...)
df.dropDuplicates([...])
df.fillna({...})
df.join(other, keys, "left")
df.groupBy(...).agg(...)
broadcast(small_df)
df.write.format("delta").mode(...).saveAsTable(...)
df.writeStream...withWatermark/window/checkpoint

```

KQL operators to remember

KQL

```

Table
| where ...
| project ...
| extend ...
| summarize Count=count(), Revenue=sum(...) by bin(Timestamp, 5m)
| join kind=leftouter (...)
| order by ... desc

external_table('ShortcutName')
| summarize ...

```

T-SQL patterns to remember

SQL

```

SELECT / JOIN / GROUP BY / HAVING
CTEs (WITH ... AS (...))
INSERT ... SELECT ...
CREATE VIEW / CREATE OR ALTER PROCEDURE
Window functions: SUM(...) OVER(PARTITION BY ... ORDER BY ...)
GRANT / DENY / REVOKE
Dynamic data masking
RLS security predicate + SECURITY POLICY
DMVs + queryinsights views
CREATE / UPDATE STATISTICS when justified

```

Appendix B - DP-700 decision cheat sheet

Question	Default mental model
Pipeline vs Dataflow Gen2 vs Notebook?	Pipeline orchestrates/moves; Dataflow is low-code Power Query shaping; Notebook is code-first Spark/custom processing.
Lakehouse vs Warehouse?	Lakehouse for open Delta/Spark/data engineering flexibility; Warehouse for relational T-SQL dimensional serving and SQL-centric governance/operations.
Eventhouse vs Lakehouse for stream?	Eventhouse for low-latency event/time-series KQL; Lakehouse for durable open Delta and broader Spark/SQL analytical reuse.
Mirroring vs Copy Data?	Mirroring continuously replicates supported source changes with low integration effort; Copy is explicit batch/data movement you orchestrate.
Shortcut vs copy?	Shortcut references data without copying; copy creates a managed duplicate and an ingestion process.
Native Eventhouse vs shortcut?	Native ingestion favors KQL performance; shortcut favors one logical copy and simpler data ownership.
Accelerated vs standard RTI shortcut?	Accelerate when OneLake data must remain authoritative but KQL performance needs caching/near-native behavior.
Full vs incremental?	Full reload replaces/reprocesses everything; incremental moves only new/changed data using file, watermark, CDC, or stream semantics.
Spark watermark?	Bounds state for late event-time data; balances correctness horizon against state/resource usage.
Optimize first or measure first?	Measure first. Use monitoring, query history, Spark plans, file layout, and activity metrics to identify the bottleneck.